The Architecture Business Cycle(ABC)

Session 4

Architecture Business Cycle (ABC)

*Software architecture is a result of **technical, business,** and **social influences**. Its existence in turn affects the technical, business, and social environments that subsequently **influence future architectures**. We call this cycle of influences, from the environment to the architecture and back to the environment, the Architecture Business Cycle (ABC).*

by **Len Bass, Paul Clements, and Rick Kazman** in *Software Architecture in Practice Book*.

Netflix Real-World Example

- Netflix originally used a monolithic system that struggled as the company grew. Several forces **influenced the architecture**, including:
  - Rapid global expansion
  - Customer demand for reliable streaming
  - Technical limits of their monolith
  - Business need for scalability and high uptime
- These influences led Netflix to create a **microservices architecture** hosted on the cloud.
- The new architecture then **influenced the organization** by:
  - Allowing faster feature releases
  - Improving system reliability and scalability
  - Enabling teams to work independently
  - Supporting business growth to hundreds of millions of users
- This success introduced new challenges and opportunities, creating **the next cycle** of architectural evolution.

Origin of the Architecture

- An architecture is the result of a set of **business, social and technical decisions**.
- Business decisions: Low Cost, Keeping people employed, invest existing corporate assets, short time to market.
- Technical decisions: Use touch screen, more functionalities, good GUI's.

Stakeholders of a system

- People and organizations who are interested in the construction of a software system.
- Examples: End users, Developers, Project manager, maintainers, marketing people.
- Different stakeholders have different expectations on the system.

Stakeholders of a System

Stakeholders Requirements (1)

- **Performance:** Accomplishment of a given task certain time period (speed).
- **Reliability:** The ability to perform its intended task without failure.
- **Availability:** Usable upon demand to perform its required function.
- **Platform compatibility:** Ability to function well on various platforms.
- **Memory utilization:** Proper utilization of system memory.
- **Security**: Protection against outside invations.
- **Modifiability:** How easy to change.
- **Usability:** Degree to which something (hardware or anything else) is easy to utilize.
- **Interoperability:** Ability to work together with other system.

Stakeholders Requirements (2)

- These properties and more determine the overall design of the architecture.
- Stakeholders may have different concerns and goals, some may be contradictory.
- Architect may need to mediate the conflicts.

Influences on Architecture by Developing Organization (1)

- Structure of the organization
  - If the organization has wealthy idle programmers skilled in client-server communications, then a client-server architecture might be used, if not, it may well be rejected.
- Budget
  - Development of some of the subsystems may be subcontracted, as subcontractors are specialized expertise and budgetary reasons.
- Nature of development
  - Immediate Business Influences.
  - Long-Term Business Influences.

Influences on Architecture by Developing Organization (2)

- Immediate Business Influences.
  - Businesses often have existing architectures, frameworks, or software products that are considered investments.
  - To maximize return on investment (ROI), these assets are often reused, adapted, or extended in the new system.
  - Example: A company with an established microservices-based architecture for its e-commerce platform may extend the same architecture to a new system for managing customer feedback. The architecture of the proposed system will naturally be influenced by the existing system's structure, tooling, and design principles.

Influences on Architecture by Developing Organization (3)

- Long-Term Business Influences.
  - Organizations often invest in foundational technologies or systems (e.g., relational databases, cloud platforms, or enterprise resource planning (ERP) systems) to support multiple projects over time.
  - These investments influence the architecture of new systems because leveraging them aligns with the organization's broader strategic goals.
  - The proposed system may not only use the existing infrastructure but also enhance and extend it for future business needs.
  - Example: A company heavily invested in Oracle Database as its relational database system will design new applications to integrate with Oracle's ecosystem. This influences the system architecture to use Oracle's tools, protocols, and best practices.

Influences on Architecture by Experience of Architect (1)

- Architects rely on their past experiences with particular architectural approaches (e.g., object-oriented or web-based).
- If a prior approach worked well, architects are likely to reuse it in future systems.
- If an approach led to poor outcomes, they are less likely to use it again.
- Experience creates biases in architectural decisions. While reusing successful patterns can save time, it might also restrict exploration of potentially better options.

Influences on Architecture by Experience of Architect (2)

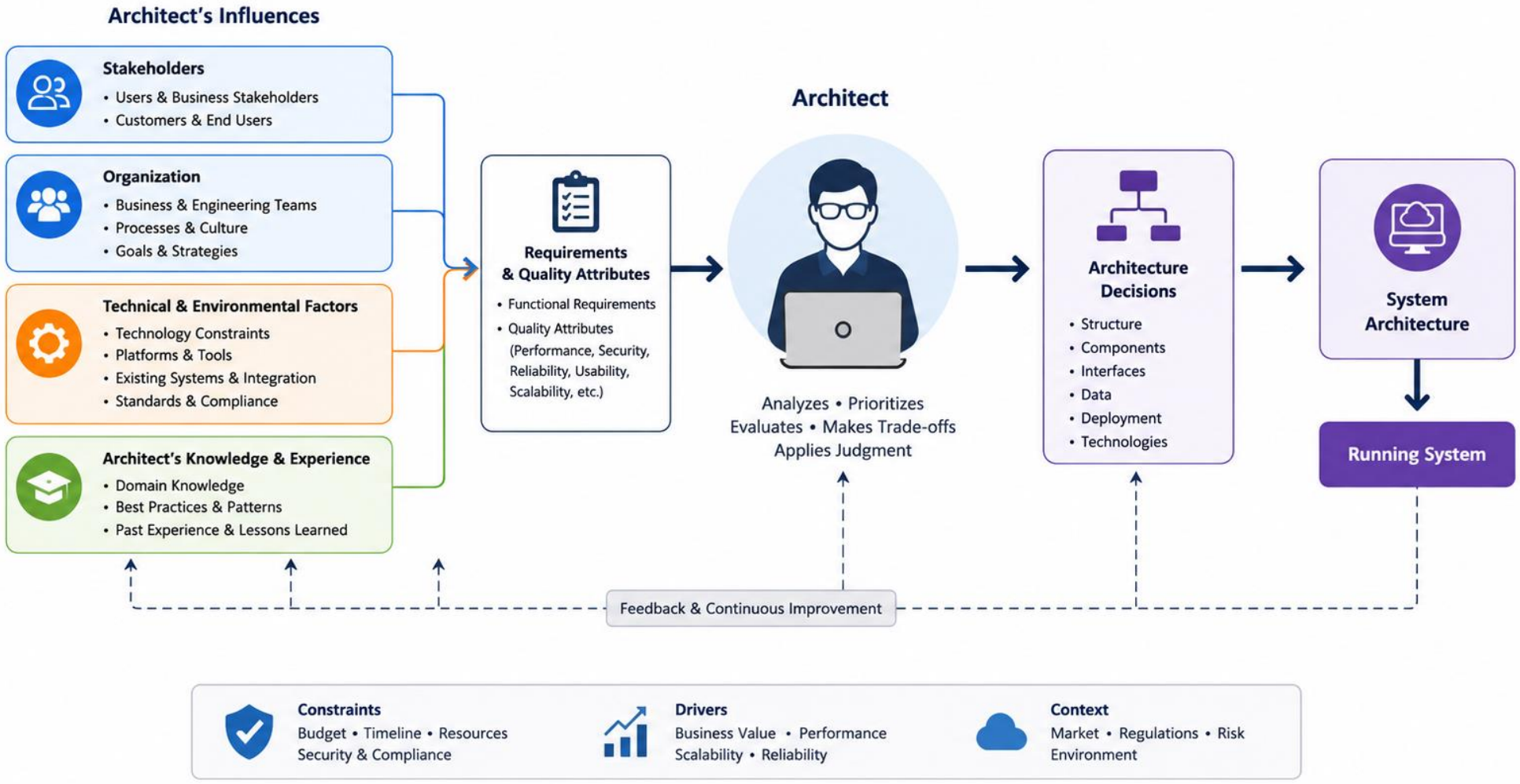
- The background, education, and training of architects shape how they approach problems and solutions.
- Exposure to Successful Patterns
  - Architects will lean toward patterns they've seen succeed.
- Exposure to Failures
  - They avoid techniques that have failed.
- Experimentation
  - Architects may try out new patterns or methods learned from books, courses, or recent training.
- Education and exposure broaden an architect's toolkit, but their choice to experiment may introduce risks or innovation. Balancing familiarity and novelty is critical.

Influences on Architecture by Technical Environment (1)

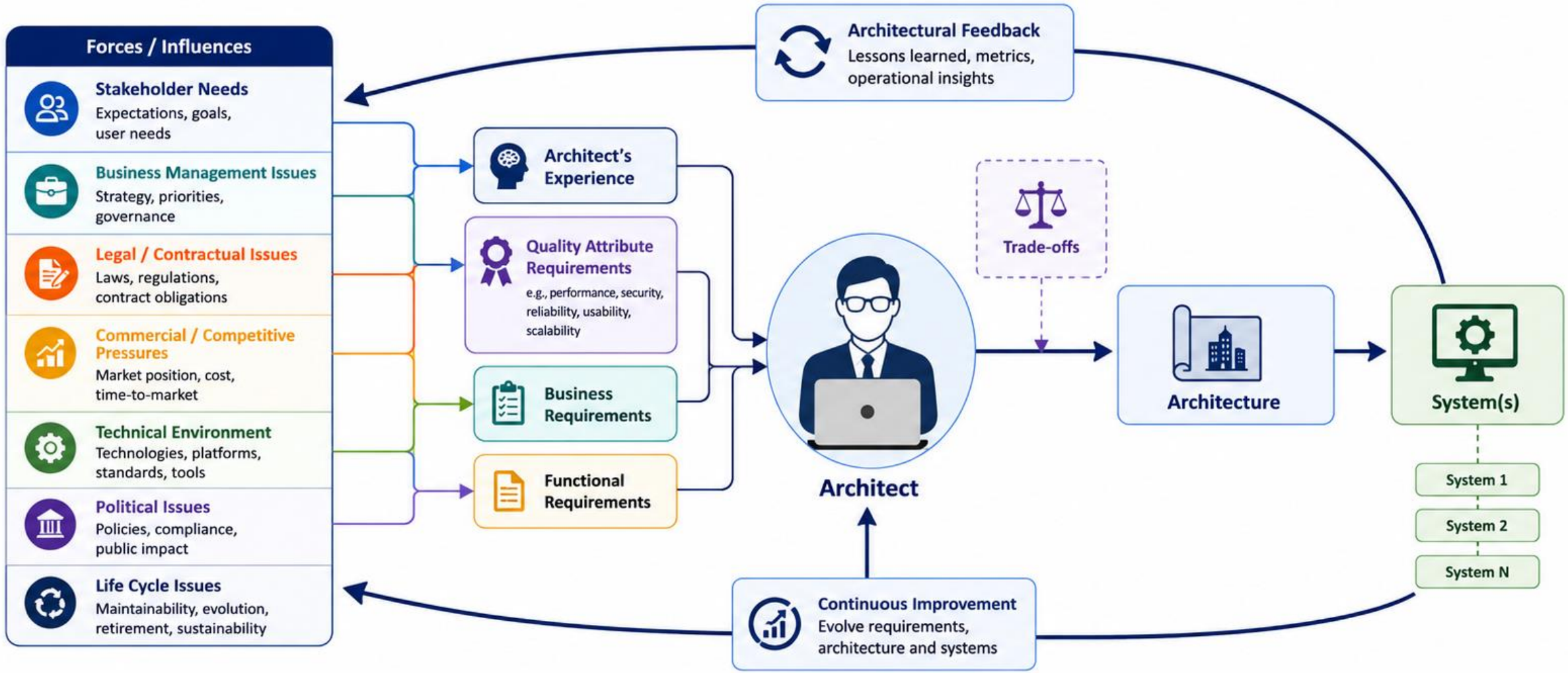
- The current technical landscape (tools, technologies, practices) strongly influences architectural choices.
  - Example: In a web-centric era, architects favor web-based, object-oriented approaches over older methods.
  - Industry standards and modern engineering practices impact architectural frameworks and methodologies.
- Architects need to stay updated with the latest technical trends. Using outdated practices may result in suboptimal architectures, while over-reliance on current trends may lead to short-lived solutions.

Influence on Architecture (1)

Architecture is shaped by multiple influences. The architect synthesizes them into effective solutions.



Influence on Architecture (2)



Aligns solutions with stakeholder goals

Manages constraints and risks

Balances trade-offs and priorities

Delivers value through sound architecture

Software Process Steps

Software process is the term given to the **organization, reutilization,** and **management** of **software development activities**.

- Steps in the Software Development Process
  - Requirements engineering process
  - Creating a software architecture
  - Using the architecture to realize a software design
  - Implementing a target system or application?
  - Testing
  - Evolution and maintenance

Steps in the Software Development Process

Requirements engineering process

- The process of **gathering, analyzing, and specifying** the **functional and non-functional requirements** of the system.
- Ensure the development team understands what the software must do and the constraints it must operate within.
- Example: Stakeholder interviews, user stories, and requirement specifications.

Steps in the Software Development Process

Creating a software architecture

- Defining the high-level structure of the software system.
- Establish **how components interact**, ensuring scalability, maintainability, and alignment with requirements.
- Example: Deciding on a microservices architecture for scalability or a monolithic architecture for simplicity.

Steps in the Software Development Process

Using the architecture to realize a software design

- Translating the **architectural vision into detailed software designs**, specifying **classes, modules, data flows, and algorithms**.
- Ensure the architecture is concretely implemented in the system.
- Example: Designing the data models, API interfaces, and specific algorithms within each architectural component.

Steps in the Software Development Process

Implementing a Target System or Application

- **Writing and integrating code** to create the functional software system.
- **Build the software according to the detailed design**.
- Example: Coding features using programming languages and integrating with databases or external APIs.

Steps in the Software Development Process

Testing

- Validating that the system meets requirements and works as intended.
- Detect and **fix bugs, ensure performance, and verify the system meets functional and non-functional requirements**.
- Example: Unit tests, integration tests, system tests, and acceptance tests.

Steps in the Software Development Process

Evolution and Maintenance

- **Updating and improving** the software **after deployment** to fix issues, add features, or adapt to changing requirements.
- Keep the software relevant, functional, and secure over time.
- Example: Deploying updates, refactoring code, or patching security vulnerabilities.

Architecture activities

- Create the business case for the system.
- Understand the requirements.
- Create or select the architecture.
- Represent and communicate the architecture.
- Analyze or evaluate the architecture.
- Implement the system based on the architecture.
- Ensure the implementation conforms to the architecture.

Architecture activities

Create the business case for the system

- Justify the **need for the system and how it will support the organization's goals**.
- Define the business drivers (e.g., profitability, market reach, operational efficiency) and constraints that influence architectural decisions.
- Assessing the market need for a system (What?)
- How much should the product cost?
- What is its targeted market?
- What is its targeted time to market?
- Will it need to interface with other systems?
- Are there system limitations that it must work within?
- Example: Identifying the expected ROI for a new e-commerce platform and its potential market impact.

Architecture activities

Understand the requirements

- There are a variety of techniques used for gathering and understanding requirements from the stakeholders.
- Object-oriented analysis uses use-cases
- Safety-critical systems use more accurate approaches, finite-state-machine.
- Creation of prototypes
  - Prototypes may help to model desired behavior, design the user interface, or analyze resource utilization.
- How much the new system differ from already existing systems?
  - It is very rare these days that system being developed is a complete new system.
  - “Requirements elicitation techniques extensively involve understanding these prior systems' characteristics.”

Architecture activities

Create or select the architecture

- Object-oriented architecture.
- Distributed systems architecture.
- Client-server architecture.
- Service oriented architecture.

Architecture activities
Communicate the architecture

- The architecture must be communicated clearly and unambiguously to all of the stakeholders.
  - Developers must understand the work assignment to them.
  - Testers must understand the task structure imposes on them.
  - Management must understand the scheduling implications it suggests.

Architecture activities
Evaluate the architecture

- Multiple Designs
  - During the architecture design process, multiple potential designs are typically created.
  - Some Designs Are Rejected Immediately
    - Designs that clearly do not meet the requirements, constraints, or goals of the system are discarded early.
    - Example: A design that cannot support required scalability might be rejected immediately for a system expected to handle millions of users.
  - Some Are Candidate Designs
    - Designs that seem feasible and align with requirements are shortlisted for further evaluation.

Architecture activities

Evaluate the architecture

- Choosing Among Competing Designs
  - Select the best design from the remaining candidates is one of the architect's toughest decisions.
  - This requires balancing trade-offs between various quality attributes and system requirements.
  - Example: An architect must decide between a monolithic design (easier to develop) and a microservices design (better scalability) based on the project’s priorities.
  - Quality Attributes
    - Performance - Response time, throughput.
    - Scalability - Ability to handle increased load.
    - Security - Protection against threats.
    - Maintainability - Ease of updates and fixes.
    - Availability -System uptime and fault tolerance.

Architecture activities

Implement the system based on the architecture

- This activity involves developers follow the structures and interaction protocols constrained by the architecture.
- The architecture provides a blueprint, including
  - The system's components.
  - Their interactions (e.g., APIs, communication protocols).Constraints (e.g., scalability, security standards).
- Developers must respect the design decisions made in the architecture phase, such as
  - Use of specific technologies or frameworks (e.g., REST APIs for communication).
  - Follow defined communication patterns (e.g., pub-sub for event-driven systems).Interaction protocols ensure that components communicate and function correctly.

Architecture activities

Ensure the implementation conforms to the architecture

- After an architecture is created and the system is implemented, the system enters a maintenance phase where it evolves over time to
  - Fix bugs.
  - Add new features.
  - Adapt to changes in technology or business requirements.
- Over time, the implementation may deviate from the original architecture due to quick fixes, ad hoc modifications, or lack of adherence to architectural principles during updates.
- Updates and changes are made in a way that respects the architectural design.
- Prevents **architectural drift** (when the system's implementation starts to diverge from the planned architecture) and **architectural erosion** (when the architecture degrades due to constant unplanned changes).
- Maintains system integrity, scalability, and performance over time.

What Makes a 'Good' Architecture?

- Different architects may design different solutions for the same requirements.
- No universally 'good' or 'bad' architecture.
- A 'good' architecture aligns with specific goals and constraints.
- Example: Distributed client-server architecture works for financial systems but is unsuitable for avionics (flight control, navigation, autopilot).

Architecture Design Guidelines

Categories:

1. Process Recommendation Guidelines: Focus on how the **architecture is designed**.
2. Structural Recommendation Guidelines: Focus on the architecture's **organization and implementation**.

Process Recommendation Guidelines

- Involve a single architect or small team with a clear leader.
- Capture functional requirements and qualitative properties.
- Document architecture with standardized notations.
- Follow a structured architecture design method (e.g., ADD – Attribute-Driven Design).
- Use iterative and incremental design instead of big upfront design.
- Validate and review design decisions regularly.
- Use prototypes or models to test architectural risks.

Process Guidelines for Resources

- To ensure the architecture is designed:
  - Systematically
  - Based on requirements
  - With stakeholder input
  - With reduced risk

"How should we design the architecture?"

Structural Recommendation Guidelines

- Design functional modules using Separation of Concerns (SoC).
- Each module should have a well-defined interface.
- Use architectural tactics to meet quality attributes.
- Avoid dependency on specific tools or products; ensure easy migration to alternatives.
- Apply appropriate architectural patterns (microservices, layered, event-driven).
- Maintain consistency in naming, styles, and coding standards.
- Prefer reusable and maintainable structures.

What is Separation of Concerns?

- Dividing a system into independent features or modules to minimize overlap.
- Features
  - Modularity: Break the system into functional parts.
  - Encapsulation: Hide internal workings of modules.
  - Information Hiding: Expose only necessary details.
- Benefits: Improves maintainability, scalability, and clarity.

Achieving Quality Attributes

- Tactics for Quality:
  - Fault tolerance through redundancy.
  - Scalability via distributed architecture.
  - Maintainability using modular design.
- Choose specific tactics based on the system's quality requirements.

Task

- Identify and explain how the following factors influence software architecture in a real-world system (e.g., an e-commerce platform, a mobile banking app, or an IoT system)
  - Business Goals
    - Identify specific goals (e.g., scalability, time-to-market) and explain how they shape the architecture.
  - Stakeholder Needs
    - Identify different stakeholders and their requirements (e.g., end-users, developers, business leaders).
  - Technical Environment
    - Explore existing technologies and constraints that would influence the architecture.